

第一次作业-吃豆人跨越迷宫

顾运-2026秋

联系方式:

yungu@ieee.org

这次作业里废话比较多，大家根据自己的实际情况阅读。

ChatGPT-101: 如何理解一个LLM的基本运行逻辑?

这一段是纯纯的科普，没兴趣可以跳过(当然，也不会考)。

另外注意，并不是所有的GPT-like模型都遵循统一的实现逻辑，我们以下只是提供一个非常非常基本的解释。

我们日常中使用的最多的是一个ChatGPT-like系统，即ChatGPT类型的对话系统，其核心技术都是大语言模型（Large Language Model, LLM）。他们遵循的思路都非常类似，用户输入提示词（即Prompt），比如我在下图中输入“上海交大是什么？”，一个基本的运行逻辑如下：

上海交通大学（简称“上海交大”）是中国历史最悠久、声誉最卓著的高等学府之一。它位于上海市，是教育部直属并与上海市共建的全国重点大学，位列国家“双一流”（世界一流大学和一流学科）、“985工程”、“211工程”重点建设高校。

关于上海交大的几个核心点：

1. **历史沿革**：学校创办于1896年，原名“南洋公学”，是中国近代高等教育的发端之一。经过百余年的发展，现已形成以理工科见长，同时拥有强大医学、经管、人文、法学等多学科的综合性格局。
2. **学术实力**：上海交大在工程技术、船舶海洋、计算机科学、生物医学、商学（安泰经济与管理学院）和医学（医学院）等领域处于国内顶尖水平。其科研经费和成果常年位居全国高校前列。
3. **校区分布**：目前主要有五个校区，包括徐汇校区（历史最悠久的校区）、闵行校区（主校区，面积巨大）、黄浦校区（医学院）、长宁校区（继续教育学院）以及临港校区（部分科研机构）。
4. **社会地位**：与复旦大学并称为上海高校的“双子星”，在国内外各类大学排行榜中通常稳居中国内地高校前10名，也是众多学子向往的顶尖学府。

Tokenization & Embedding：分词+嵌入

什么是分词呢？顾名思义，用户输入的提示词通常由若干个基本的单词组成，比如英语里面通常单词之间通过空格隔开，中文里面也有类似的机制。比如“上海交大是什么？”是由“上”，“海”，“交”，“大”，“是”，“什”，“么”，“？”这8个基本汉字（标点符号）构成。大语言模型通过分词器（Tokenizer）会对于上述的汉字进行组合，组合的依据是常见词的词表，比如“上海”是一个高频词，“交大”是一个高频词，分词器会根据高频词词表，将上面的提示词拆分为：“上海”，“交大”，“是”，“什么”，“？”。这里需要注意，分词过程与当前LLM的具体实现高度相关。比如有的LLM会将上面的“上海”和“交大”合并。

完成分词后，LLM模型会执行“嵌入”（Embedding）。嵌入是人工智能领域非常重要的一个概念，比较有代表性的一个工作是Word2Vec。人类长久以来都非常关心一个问题，我们应该怎么定量地描述文字之间的关系呢？大家都在大一的课程里学过ASCII码，这是一种对于单个字符的编码系统，比如大写字母“A”的ASCII码是65，小写字母“a”是97。而为了编码更为复杂的字符集，人们后面又发明了Unicode，实现了对于世界范围内所有文字的编码。但是，无论是ASCII还是Unicode，都是对于单个字符的编码，无法描述更为复杂的语义信息。

嵌入的作用：比如给定3个词，“爸爸”，“妈妈”，“植物”。我们有个很直观的感觉，“爸爸”和“妈妈”都是人类对于父母的称呼，描述了某种关系。而“植物”则与上述两个单词似乎没什么关系。因此，“爸爸”和“妈妈”这两个词之间的关系相比于“植物”更加近一些？如果我们存在某种距离度量 d ，那么我们会有：

$$d(\text{"爸爸"}, \text{"妈妈"}) < d(\text{"爸爸"}, \text{"植物"})$$

即“爸爸”和“妈妈”这两个词之间的距离相比于“植物”会更近一些，或者说，二者在语义上的相似度应该更相似一些。很遗憾，Unicode编码搞不定这个过程。因此，研究者提出了嵌入技术。嵌入的思路很简单，给定任意一个词，都会对应一个 n 维的词向量，那么两个单词之间的关系可以通过计算这两个词向量之间的距离（比如欧式距离或者汉明距离）实现。

综上，通过分词+嵌入，用户给定的提示词会被拆解为若干个 n 维向量。

Next Token Prediction：下一个词的预测

好的，此时我们把用户的输入理解完毕了，那么怎么执行输出呢？其实输出的过程非常简单。还记得上面我们提到过，LLM会维护一个词表，覆盖了所有可能的输出单词。那么ChatGPT或者DeepSeek的任务就是：从个词表里挑出一个“最合适”的单词，然后输出。那个“最合适”的单词是怎么挑出来的呢？实际上这个取决于LLM在训练阶段所见过的语料了。

所有的LLM都不是无中生有。他们的智慧都来源于“阅读”了大量现有的语料（比如书籍，网页，源代码，微博，知乎问答，小红书等等）。假设现有的语料里，和“上海”，“交大”相关的内容如下：

上海交大牛逼！<出现了90次>

上海交大厉害！<出现了9次>

上海交大强！<出现了1次>

那么就意味着，现有的语料告诉你，当我们给定“上海”，“交大”这两个词，可以得到一个粗略的统计信息：

$$p(\text{"牛逼"}|\text{"上海"}, \text{"交大"}) = 90 / (90 + 9 + 1) = 0.9$$

$$p(\text{"厉害"}|\text{"上海"}, \text{"交大"}) = 9 / (90 + 9 + 1) = 0.09$$

$$p(\text{"强"}|\text{"上海"}, \text{"交大"}) = 1 / (90 + 9 + 1) = 0.01$$

$$p(\text{词表中的其他词}|\text{"上海"}, \text{"交大"}) = 0$$

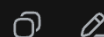
大语言模型基于海量语料，学习上述条件概率分布。那么在给定任意输入，我们都可以预测输出词表中任意词的概率，概率越大则输出的可能性越高。这里需要注意的是，上述过程是一个粗略的模型，具体涉及到的细节非常多，在此就不过多展开了。

我们还需要回答下一个问题，输出第一个词过后，第二次怎么来呢？这个过程很简单，那就是把条件扩充。比如我们给定问题：“上海交大是什么？”，模型输出第一个词是“上海”，那么第二次的预测过程则是在给定“上海交大是什么？上海”作为条件，预测下一个词是什么。

Thinking

我们在与大模型之间的交互过程经常能见到一种有意思的行为，大模型似乎在自言自语，脑洞大开？

上海交大是什么？



Thought for 3 seconds

- 嗯，用户询问“上海交大是什么？”。

这是一个简单的定义性问题。用户可能想了解上海交通大学的基本信息，比如它的性质、地位和特点。

我需要给出一个清晰、简洁的概述，说明它是一所什么样的大学，包括其简称、地理位置、办学层次、历史渊源、学术地位和主要特色。回答时应当直接、准确，无需展开过多细节。

想到了可以从这些方面组织信息：先点明其全称和简称，说明是一所顶尖大学，然后简要提及历史、地位和优势学科。

上海交大，全称上海交通大学，是中国历史上最悠久、享誉海内外的著名高等学府之一。

具体来说，它有以下几个关键点：

1. **办学性质**：它是中华人民共和国教育部直属、并由上海市共建的全国重点大学。
2. **顶尖地位**：它属于中国高等教育的第一梯队，是“985工程”、“211工程”首批重点建设高校，2017年首批入选国家“双一流”（世界一流大学和一流学科）建设高校名单（A类）。

这种策略叫做Thinking，用于解决大语言模型在直接回答问题时缺乏逻辑造成回答不完整或者质量不高的问题。

小结

一个ChatGPT-like系统，就是通过上述的分词，嵌入，预测过程完成人机之间的对话。

VibeCoding-101：简单尝试

如果你有丰富的Vibe Coding编程经验，这一段可以跳过

你可以在各式各样的平台看到如何基于大语言模型(Large Language Model, LLM)或者Vibe Coding(aka 氛围编程)的教程。我们这里梳理两种最常用的手段：基于网页版LLM的对话 & 基于API的IDE环境编程。

网页版LLM

如果你在大一上C++课程（CS1501 程序设计思想与方法，碰巧我也上这门课）时候，可能因为EduCoder而痛苦过？不过在LLM看来，EduCoder那点题目还是太easy了。一个例子：

任务描述

本关任务：读入两个正整数 a 和 b ，并且 $a \leq b$ ，计算并输出闭区间 $[a, b]$ 中的X数。X数为各个数位上数字的立方和等于其自身的数字，例如：

$153 = 1 * 1 * 1 + 5 * 5 * 5 + 3 * 3 * 3$ // 153 is a X-number.

12 is not equal to $1 * 1 * 1 + 2 * 2 * 2$ // 12 is not a X-number.

如果没有找到X数，输出提示信息no

编程要求

根据提示，在右侧编辑器补充代码，计算并输出相应的结果。

测试说明

平台会对你编写的代码进行测试：

测试输入: 1 200

预期输出:

1 153

测试输入: 10 100

预期输出:

no

最简单的做法，那就是把上面这段话直接丢给DeepSeek（任何你觉得靠谱的LLM都可以）

C++ code for X numbers

我现在需要写一个C++代码，完成以下需求: "##### **任务描述**

本关任务: 读入两个正整数a和b, 并且 $a \leq b$, 计算并输出闭区间[a,b]中的X数。X数为各个数位上数字的立方和等于其自身的数字, 例如:

153 = $1 * 1 * 1 + 5 * 5 * 5 + 3 * 3 * 3$ // 153 is a X-number.

12 is not equal to $1 * 1 * 1 + 2 * 2 * 2$ // 12 is not a X-number.

如果没有找到X数, 输出提示信息no

****编程要求 ****

根据提示, 在右侧编辑器补充代码, 计算并输出相应的结果。

****测试说明****

平台会对你编写的代码进行测试:

测试输入: 1 200

预期输出:

1 153

测试输入: 10 100

预期输出:

no"

DeepSeek非常快, 给了你一个答案:

```
#include <iostream>
using namespace std;
```

```

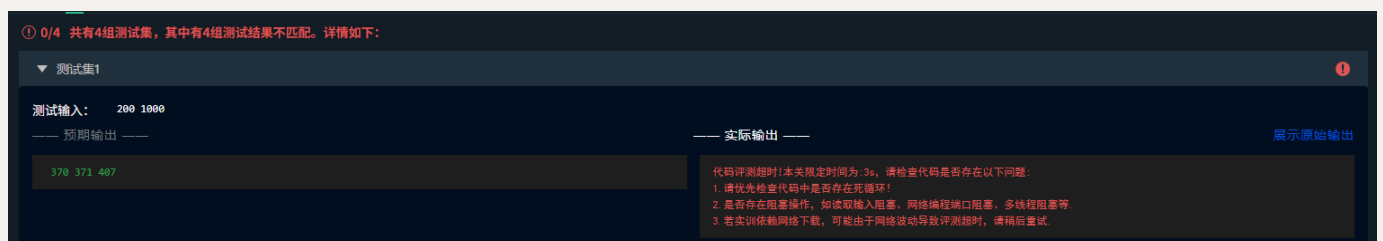
int main() {
    int a, b;
    cin >> a >> b;

    bool found = false;
    for (int i = a; i <= b; i++) {
        int sum = 0, temp = i;
        // 计算各位数字的立方和
        while (temp > 0) {
            int digit = temp % 10;
            sum += digit * digit * digit;
            temp /= 10;
        }
        // 判断是否为x数
        if (sum == i) {
            if (found) cout << " "; // 已有结果，先输出空格
            cout << i;
            found = true;
        }
    }

    if (!found) cout << "no";
    return 0;
}

```

然后你很高兴地把这段代码贴给EduCoder，喜提超时。



好的，不要气馁，请把这些输入输出统统告诉大模型。

你太笨了，大一的题目都做不对，

测试输入：

200 1000

—— 预期输出 ——

370 371 407

—— 实际输出 ——

代码评测超时!本关限定时间为:3s, 请检查代码是否存在以下问题:

1.请优先检查代码中是否存在死循环!

2.是否存在阻塞操作, 如读取输入阻塞、网络编程端口阻塞、多线程阻塞等.

3.若实训依赖网络下载, 可能由于网络波动导致评测超时, 请稍后重试.



Thought for 30 seconds

- 我们被用户指责了，说代码超时。但提供的代码很简单，不会超时。可能用户把代码提交到某个在线评判系统，测试输入200 1000，预期输出370 371 407，但实际输出超时。但代码本身没问题。也许是因为用户没有把代码放在正确的框架里？或者系统要求必须使用某个函数？用户说“大一的题目都做不对”，可能他们期望的代码格式有所不同。但给出的代码逻辑正确，不会超时。可能系统要求从某个特定函数读取？也可能是输出格式问题：预期输出前面有空格？预期输出是“ 370 371 407”，前面有一个空格，但通常我们输出“370 371 407”也没问题。但超时说明代码可能死循环？但代码中for循环范围明确，不会死循环。也许用户复制时漏了头文件？或者编译环境问题？我们得分析一下。

可能的原因：输入范围 $a \leq b$ ，但用户给的测试200 1000，数字较大，但计算量很小，不会超时。但系统说超时，可能因为代码中包含了死循环？仔细看代码，while (temp > 0) 循环，temp会不断除以10，最终为0，正常。没有死循环。

也许用户使用的系统需要处理多个测试用例？但题目描述只是读入两个正整数。可能系统会连续输入多组？但任务描述没有说多组。但实际评测时可能有多组测试数据，代码应该循环读取直到EOF。但题目没说。很多OJ题目会要求处理到文件尾。如果输入有多组，而我们只读一组，就会导致后续读取阻塞，等待输入，从而超时。这可能是真正原因。系统可能有多组测试输入，但我们的代码只处理一组，然后程序结束，但系统可能还在等待输出，导致超时？实际上，如

很可惜，还是超时。那么我们应该如何与LLM更好地对话呢？你需要与当前的LLM持续对话下去，终于发现了一些问题，其实目前我所对话的LLM还是没能正确理解题目。

Thought for 69 seconds ▾

```
return 0;
}
```

如果还是超时，建议用户检查代码运行环境是否有其他限制，或者题目是否要求使用函数。也可能评测系统对每行输出要求无换行？但一般都有换行。

最后，我怀疑用户实际超时是因为他直接复制了我们的代码，但代码中有死循环？不，没有。

我们尝试一种极端优化：使用预计算的立方表，并在循环内使用整数除法，但差异很小。

也许题目中 a 和 b 的范围很大，比如 1 到 10^9 ，那么循环会超时。但题目没有说明，但通常水仙花数只有几个，可以先生成所有水仙花数，然后判断区间内的。但题目要求的是“X数”，即水仙花数（阿姆斯特朗数），在十进制下，只有有限个：1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407, 1634, 8208, 9474, 54748, 92727, 93084, 548834, 1741725, 4210818, 9800817, 9926315, 24678050, 24678051, 88593477, 146511208, 472335975, 534494836, 912985153, 4679307774, ... 但题目只要求正整数 a, b ，且 $a \leq b$ 。如果范围很大，遍历每个数会超时。所以更优的方法是先列出所有水仙花数（在 32 位整数范围内最多几十个），然后输出区间内的。

考虑到这是一个大一的题目，通常不会要求这么大的范围，但以防万一，我们可以采用预先生成所有水仙花数的方案，这样即使 a, b 很大，也能快速输出。

我们可以提前计算所有水仙花数并存储在数组中，然后对于每个查询，遍历数组输出在区间内的。

下面给出一个使用预先生成所有水仙花数的代码，可以处理任意大范围（但限于 32 位 int 范围内）。这样即使 a, b 是 1 到 10^9 ，也能瞬间完成。

cpp

Copy

Download

```
#include <iostream>
```

比如上图中的红框内容，显然不是完全正确的，因此，我们需要引导LLM意识到错误并进行修正。

Note

LLM的确很强大，但是输出结果未必靠谱，请一定要记得刨根问底。

一个简单的本地LLM-Assisted IDE

刚才我们大致通过一个与DeepSeek模型之间的问答过程了解了基本的LLM-Assisted编程逻辑，但是目前这个版本还是难以满足我们的复杂编程需求。特别是涉及到多个文件协作，本地调试等必要操作时，基于网页的LLM Coding还是比较麻烦。因此，我们可以考虑将LLM集成到本地的开发环境中。目前主流的选择非常多，比如CODEX与Claude Code（这两个工具很强，但是需要一点点门槛&金币，如需使用这两个工具，请自行在网上搜索使用方法）。我们课程上介绍基于VSCode的一个插件Cline，配置简单并且可以与市面上的诸多大

模型API（交大也提供了校内的API服务）直接整合。注意：可以在国庆假期多尝试一些 Vibe Coding 的方案！

如何从交大申请免费的大模型资源？

1. 登录交我办申请 API-KEY

- (1) 登录交我办网页端 <https://my.sjtu.edu.cn/>，或移动端 APP
- (2) 搜索【“致远一号”AI模型API申请（内测）】→ 点击流程申请大模型API

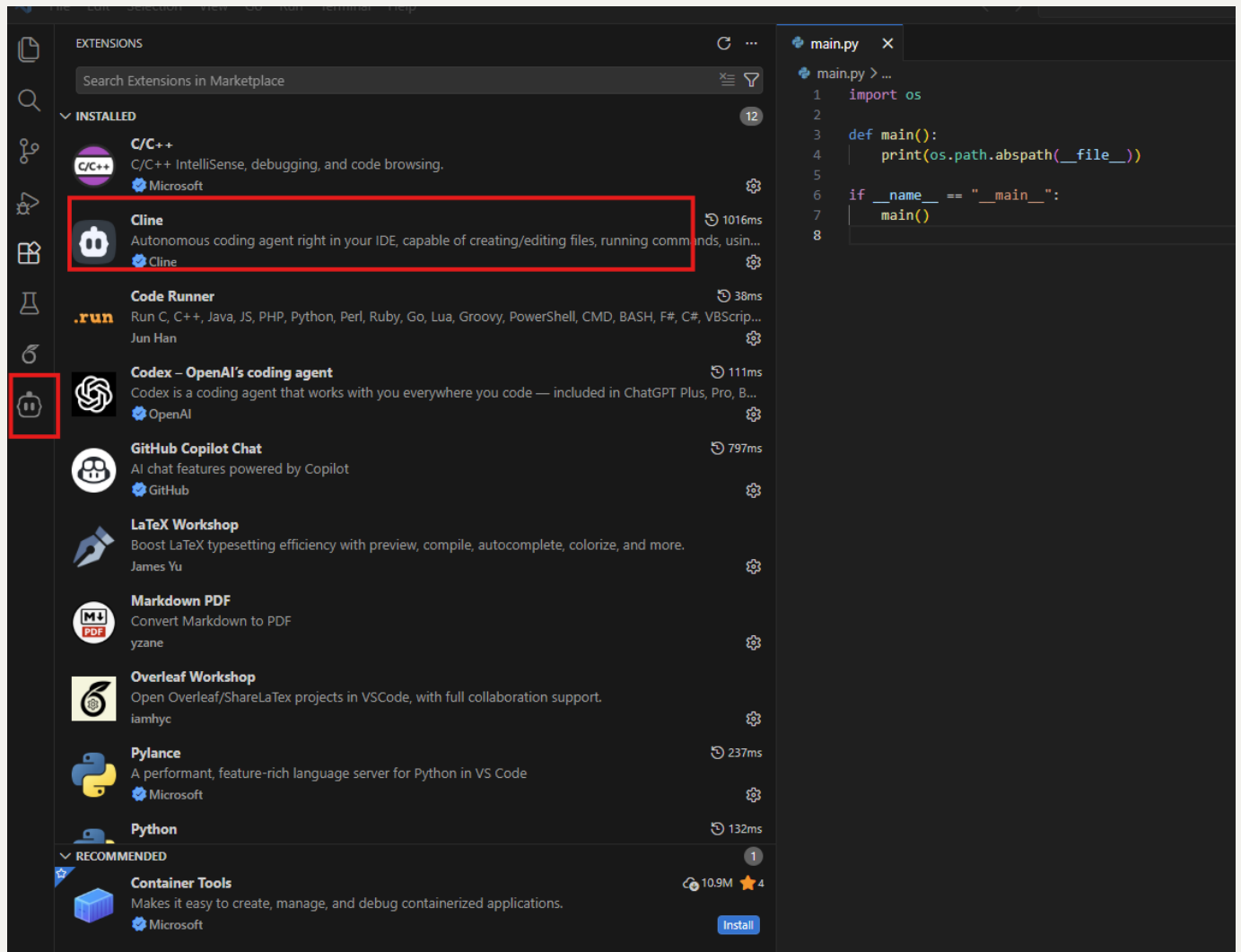


- (3) 填写申请表 → 提交申请

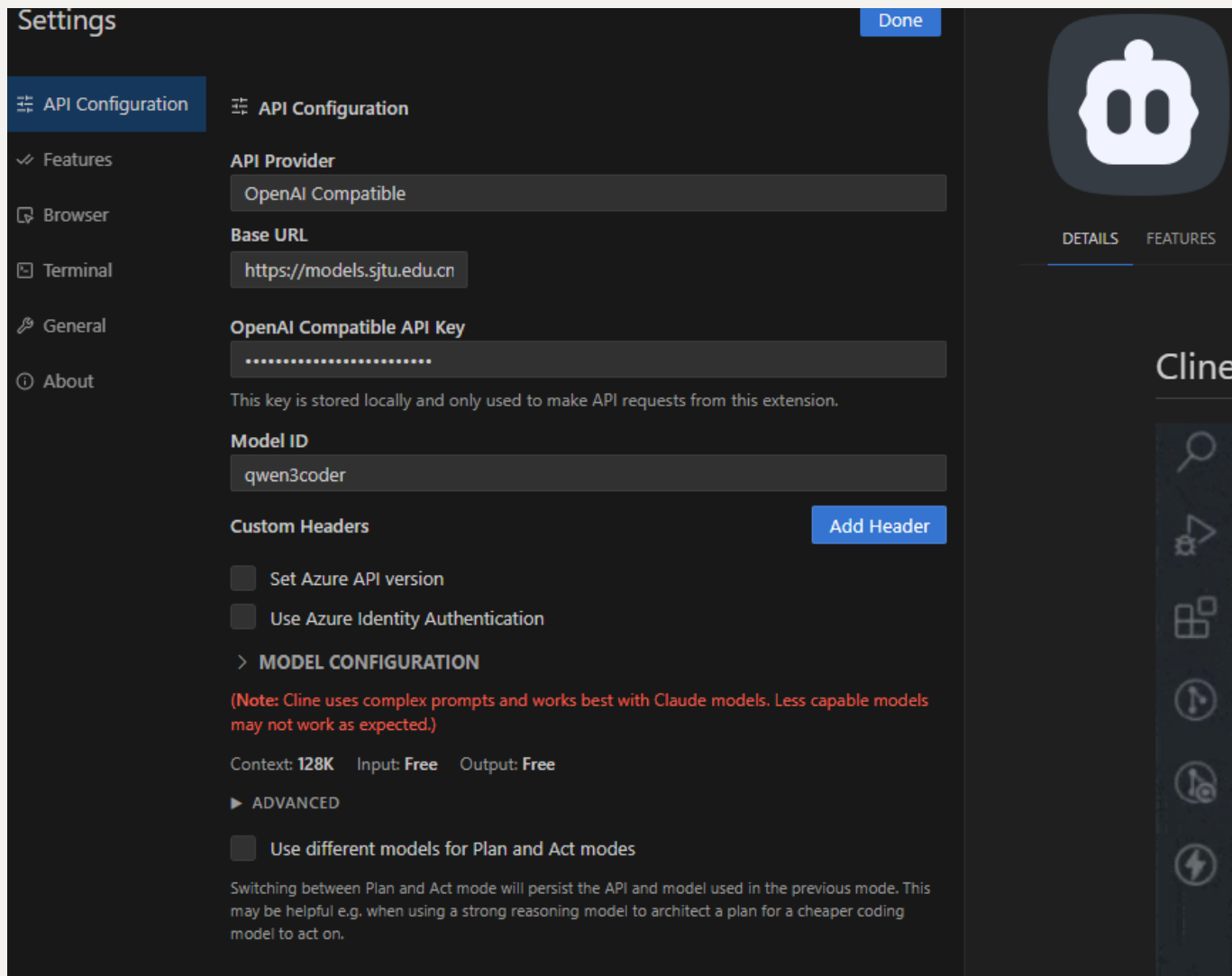
- **联系邮箱：**填写用于接收开通信息的邮箱
- **模型：**目前提供 `DeepSeek-R1-685B`、`DeepSeek-V3-685B`、`Qwen3-Coder-30B`、`Qwen3-VL-32B` 四种模型服务
- **默认 token 有效期：**截止到 2025-12-31（内测结束）
- **默认 token 额度：**
 - 教职工：每分钟请求数 300，每分钟 token 消耗 3000，每周 token 总量 30,000,000
 - 学生：每分钟请求数 100，每分钟 token 消耗 1000，每周 token 总量 1,000,000
- **用途说明：**请大致填写使用目的及具体应用场景

- (4) 申请通过后会系统填写的邮箱及交我办消息中收到 `base_url` 和 `api-key`

在VSCode里面安装Cline



在VSCode扩展商店里搜索Cline并安装，安装成功后可以在左侧看到上图中的图标，点击后可以看到套餐选择。我们可以选择第一种免费的（比如Minimax的免费API），也可以选第二种付费版本。如果你已经拿到了交大的大模型API或者其他第三方提供的API，那就选择第三个自定义。



在自定义界面，

API Provider请选择：OpenAI Compatible，

BaseURL如果你选择使用交大的API，则填写：<https://models.sjtu.edu.cn/api/v1/>（注意，不是你邮件收到的那个<https://models.sjtu.edu.cn/api/v1/chat/completions>）

OpenAI Compatible API Key则填写你在交大邮箱里收到的api_key。

注意，如果你使用交大API，那么需要保证你连接的是交大校园网或者使用了交大VPN。

Note

目前有非常多的LLM可供选择，大家可以多尝试，感受一下不同LLM的效果

开始第一次VibeCoding

假设我们现在目录下有一个main.py，内容如下：

```
import sys

def bubble_sort(arr: list[int]) -> list[int]:
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def selection_sort(arr: list[int]) -> list[int]:
    raise NotImplementedError("Selection sort is not implemented yet.")

def readCommand(argv):
    from optparse import OptionParser
    usageStr = """
    USAGE:      python main.py <options>
    """
    parser = OptionParser(usageStr)
    parser.add_option('-s', '--sort', dest='sort', type='str',
                      help='The sorting algorithm to use: bubble or selection',
                      default='bubble')

    options, otherjunk = parser.parse_args(argv)
    args = dict()
    args['sort'] = options.sort

    if len(otherjunk) != 0:
        raise Exception('Command line input not understood: ' +
                        str(otherjunk))
```

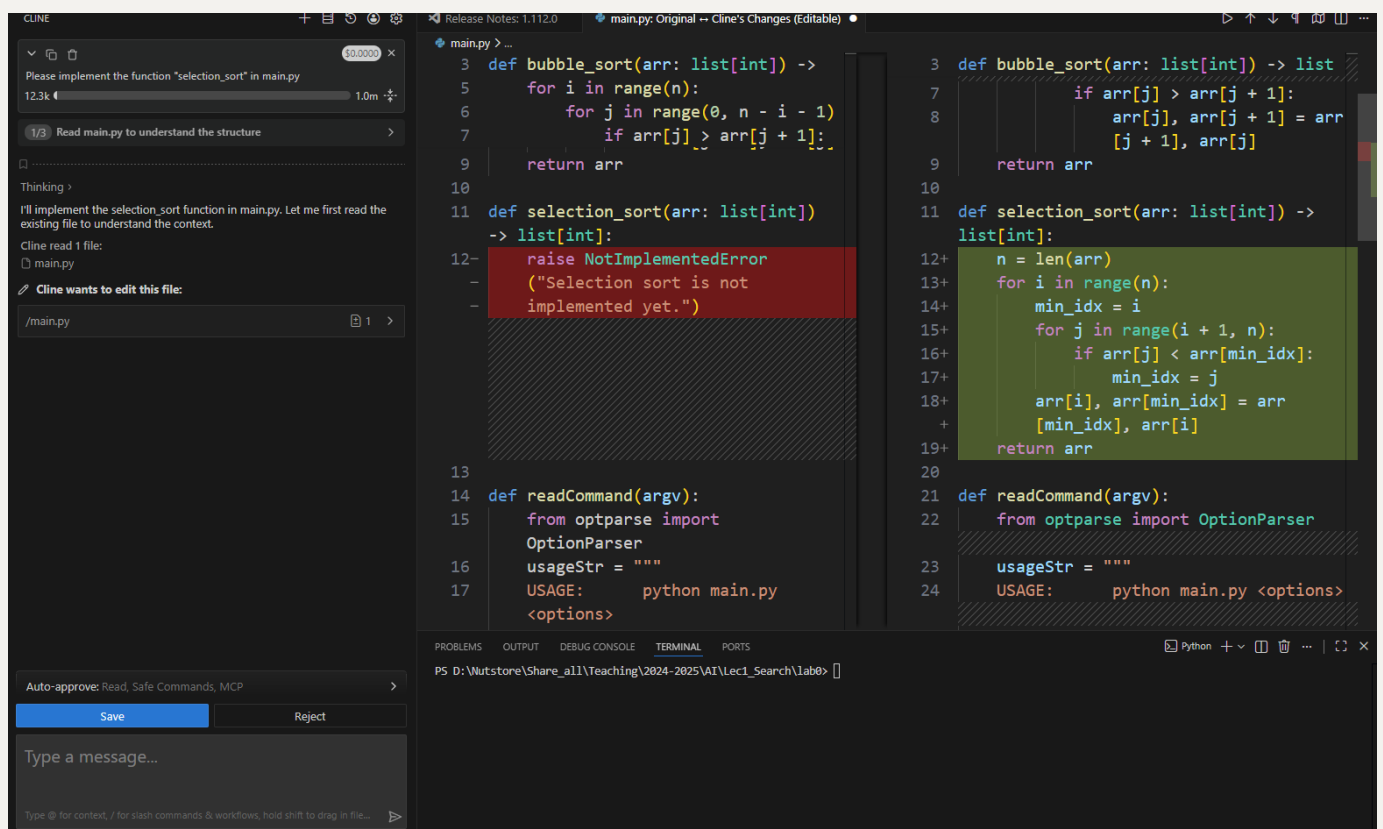
```

return args

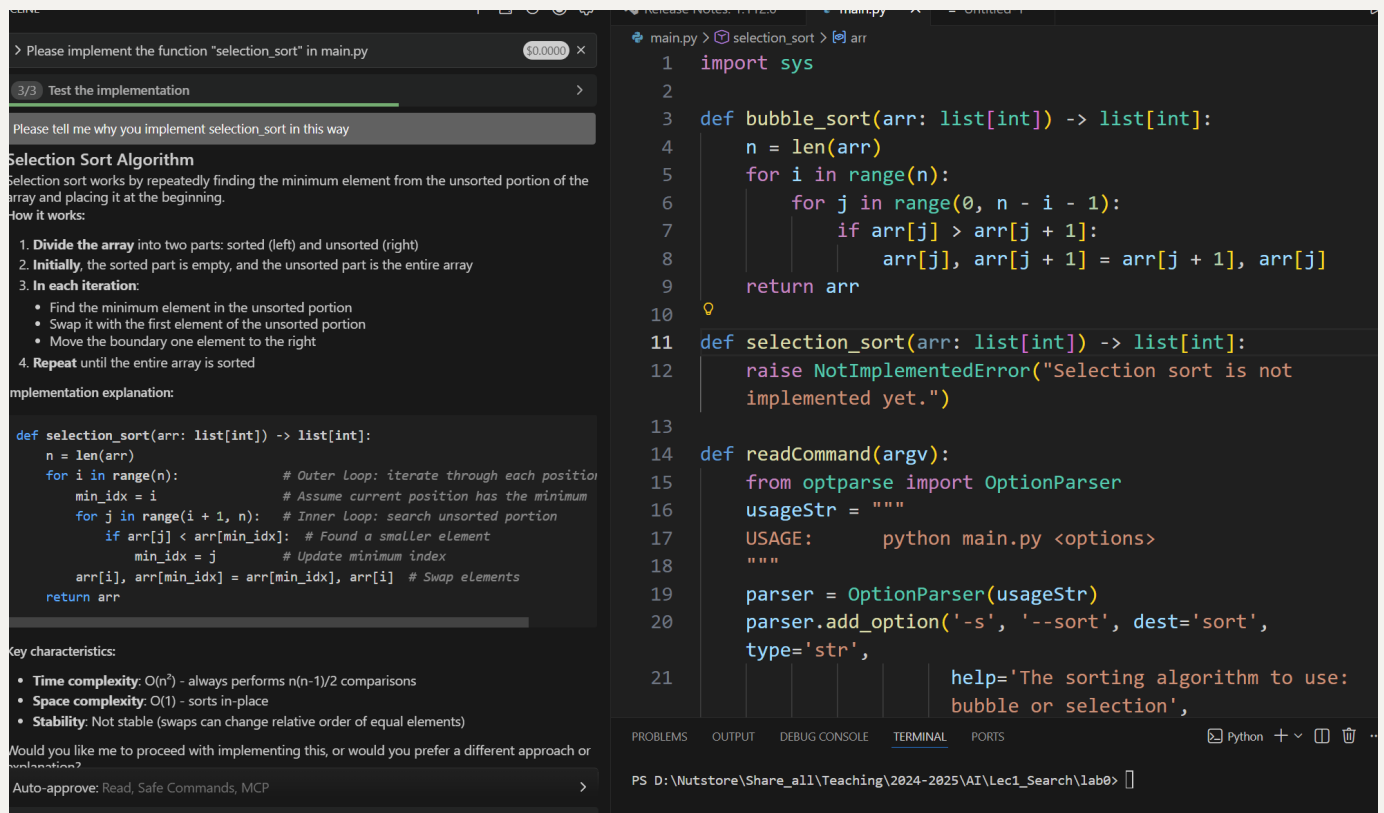
if __name__ == "__main__":
    opt = readCommand(sys.argv[1:])
    data_to_sort = [64, 34, 25, 12, 22, 11, 90]
    if opt['sort'] == 'bubble':
        print(bubble_sort(data_to_sort))
    elif opt['sort'] == 'selection':
        print(selection_sort(data_to_sort))
    else:
        print(f"Unknown sorting algorithm: {opt['sort']}")

```

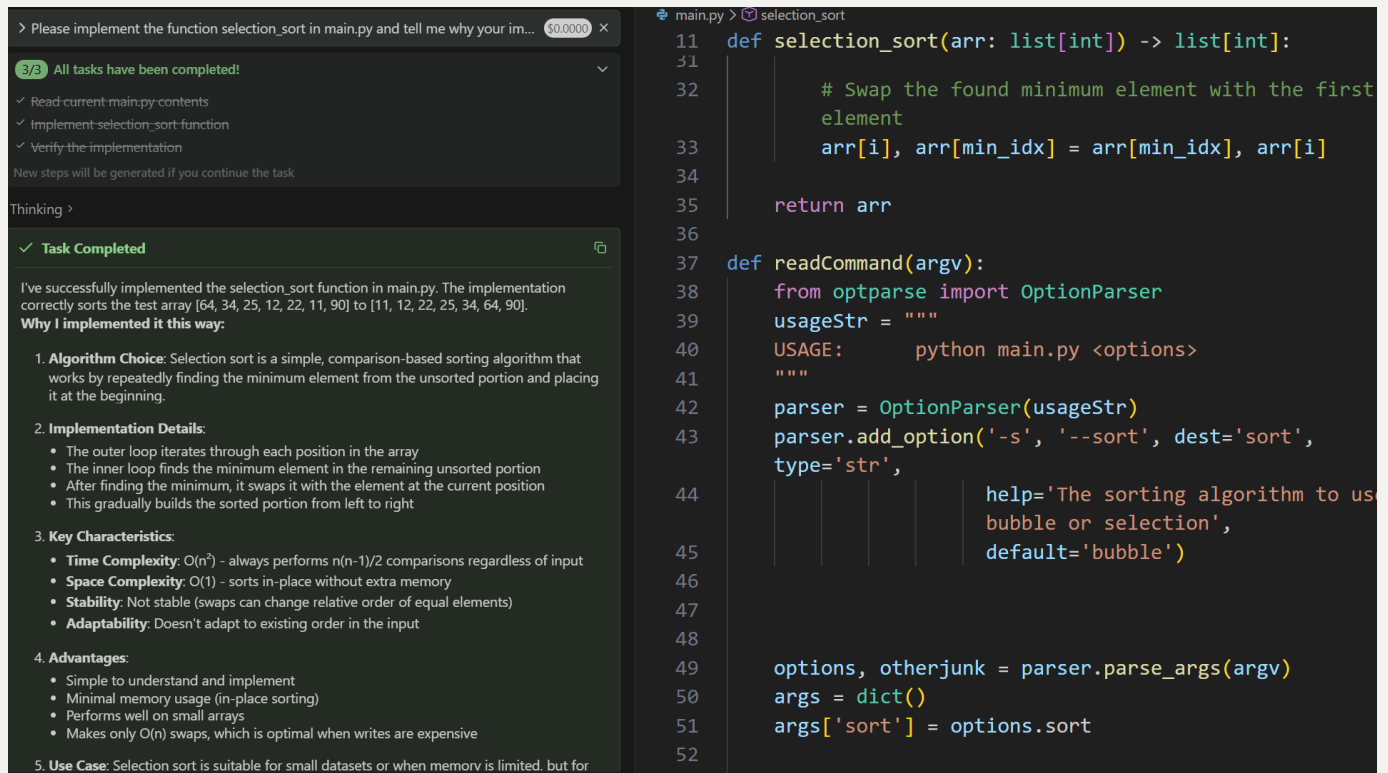
我们后续的所有作业都是以上述形式，需要大家根据要求完成一个函数或者一个类。我们可以尝试让Cline替你完成这个任务。



比如，我们在Cline的“Type a message”中输入：“**Please implement the function "selection_sort" in main.py**”。Cline很快就为你生成了一个版本，如上图所示，红色高亮处是原始的selection_sort函数的代码，右侧黄绿色高亮处为修改后的代码。如果你觉得内容OK，直接点击“Save”即可。此时，Cline会为你自动替换。



在此基础上，继续追问当前代码的实现细节，比如“Please tell me why you implement selection_sort in this way”。可以看到，当前LLM给出了比较详细的解释，包括选择排序算法的基本原理，代码执行过程以及一些关键时空复杂度的分析。当然，这个都只是问答的过程。Cline还能够支持更为复杂的操作，当你选择Save后，Cline会自动测试当前代码的正确性。



可以看到，Cline自动调用了本地的Python解释器，执行了当前的代码，并根据预期输出与真实输出的匹配关系确认代码的正确性。

看来，EduCoder那点题目还是太easy了。一个例子：

Note

- 当然，这里有可能存在一些问题，比如Cline未必正确调用了你当前虚拟环境的Python解释器，那这个该怎么解决呢？如果这个任务是一个图形界面程序，又该如何解决呢？请思考并尝试之。
- 如果大语言模型修改了代码，但是执行的结果并不正确，我们应该如何解决？
- 如果我们的作业要求放在了README.md中，如何让LLM直接读取README并完成？

终于.....开始第一次作业的介绍

打开第一次作业的Project，你可以通过以下方式运行：

```
python main.py -l MAP_NAME -a fn=YOUR_ALGOR
```

其中，**MAP_NAME** 是当前任务的地图，你可以选择：

```
tinyMaze
mediumMaze
```

关于地图的定义，你可以在"layouts"目录下找到. 地图内包含如下元素：

- 吃豆人(P): 吃豆人每走一步都会扣1分；
- 终点(.): 到达终点就结束；
- 墙(%): 吃豆人无法穿越墙
- 陷阱(T): 如果吃豆人经过了陷阱，会扣去大量分数。

YOUR_ALGOR 是当前搜索算法，你可以选择：

```
dfs
bfs
ucs
astar
```

例如，你希望采用深度优先搜索算法，你可以执行：

```
python main.py -l tinyMaze -a fn=dfs
```

观察终端的输出结果，你可以看到，采用深度优先搜索算法最终的累积回报（Cost）为108，搜索过程扩展了15个节点。我们这次作业的要求就是：如何在扩展节点数目尽可能少的情况下（快），得到更高的累积回报（优）？

```
[SearchAgent] using function dfs
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 108 in 0.0 seconds
Search nodes expanded: 15
```

请注意，如果你使用的A*算法，那么我们需要提供相应的启发函数，格式如下：

```
python main.py -l mediumMaze -a
fn=astar,heuristic=manhattanHeuristic2
```

其中，"manhattanHeuristic2" 是我们在 "heuristics.py" 文件中实现的函数。你也可以在 "yourHeuristic" 中实现你自己的启发式函数。

```
def yourHeuristic(position, problem, info={}):
    return 0
```

然后运行：

```
python main.py -l mediumMaze -a fn=astar,heuristic=yourHeuristic
```

请完成search_func.py中的函数：

```
def breadthFirstSearch(problem):
    """Search the shallowest nodes in the search tree first."""
    """ *** YOUR CODE HERE *** """
    raiseNotDefined()

def uniformCostSearch(problem):
    """Search the node of least total cost first."""
    """ *** YOUR CODE HERE *** """
    util.raiseNotDefined()

def aStarSearch(problem, heuristic=nullHeuristic):
    """Search the node that has the lowest combined cost and
    heuristic first."""
    """ *** YOUR CODE HERE *** """
    raiseNotDefined()
```

你也可以阅读"depthFirstSearch"函数的实现过程（or引导LLM），思考如何快速地基于depthFirstSearch函数的风格迁移到其他搜索算法。

我们在"util.py"中提供了一些基本的函数与类，你也可以在"external_lib.py"中添加你自己所需的函数与类. 注意，请不要在其他的修改或者添加其他py文件。请最终提交以下文件至CANVAS:

```
heuristics.py  
search_func.py  
external_lib.py
```

此外，请提交一份报告（格式不限，你可以手写，使用word，LaTeX，甚至plain text都可以），描述一下启发式函数的设计思路。另外，如果你使用了LLM辅助，请在报告里描述一下本次代码实现心得（可以是你的调用逻辑，甚至是LLM实现的思路与你的思路之间的优劣比较，没有内容限制）。

Have Fun!